

Trabajo Práctico Integrador: Gestión de Datos de Países en Python

Institución: Universidad Tecnologica Nacional

Carrera: Tecnicatura Universitaria en Programación a distancia

Materia: Programación 1

Estudiante:

- Nahuel Ismael Leyes Moyano
DNI: 47724217
N° de legajo: 54243
- Facundo Tomás Maldonado
DNI: 48049414

Fecha de entrega: 22/06/2026

Índice

Marco Teórico	2
Listas	2
Diccionarios	2
Funciones	2
Ordenamientos	3
Estadísticas Básicas	3
Manejo de archivos (CSV)	3
Decisiones Técnicas y Arquitectura	4
Diagrama de flujo del funcionamiento del proyecto:	4
Estructuras de datos principal	4
Implementación de interfaces interactivas: Questionary.....	6
Dificultades y Conclusiones	7
Dificultades Técnicas	7
Conclusión	7
Bibliografía / Webgrafía	7
Repositorio de GitHub y video explicativo del proyecto.....	7

Marco Teórico

Listas

En Python, una lista (o array) es una estructura de datos que permite almacenar múltiples elementos del mismo tipo, organizados de manera ordenada, los elementos son accesibles mediante un índice numérico que comienza en cero. Su característica principal es la mutabilidad, que la hace versátil y permite modificar, añadir o eliminar sus elementos internamente o a lo largo de la ejecución sin alterar la identidad de la propia lista.

Ejemplo:

```
lista_numeros= [] #una lista vacia para almacenar elementos, por ejemplo, números
lista_numero.append(1) #con .append() se agregan elementos de forma dinámica
lista_numeros.append(2)
print(lista_numeros) #se muestra por pantalla: [1,2]
```

Diccionarios

Un diccionario es una estructura de datos mutable que asocia claves (keys) con valores (values). Se define como una generalización de las listas donde los índices no están limitados a números ordenados, sino que pueden ser casi cualquier dato inmutable (generalmente cadenas de texto), permitiendo acceder y recuperar información mediante los valores asociados a sus claves. Importante, no pueden existir dos claves iguales, por ejemplo; "numero": 1, "numero": 2, pero si dos valores iguales.

Ejemplo:

```
#Diccionario que contiene como clave nombres de alumnos y como valores sus notas
nota_alumnos: {"Sofia": 6, "Jesús": 8}
#Se recorre el diccionario y gracias a items() podemos descomprimirlo en "alumno"
como clave y "nota" como su valor
for alumno,nota in nota_alumnos.items():
    print(f'nombre: {alumno} | nota: {nota}')
```

Funciones

Una función es un bloque de código que puede ser reutilizado, el cual realiza una tarea específica. Su propósito es permitir la repetición de esa secuencia tantas veces se requiera, sirviendo para descomponer un problema complejo en tareas más pequeñas y sencillas, en el cual pueden recibir un dato de entrada como parámetro, y de manera opcional retornar un valor o resultado como dato de salida.

Ejemplo:

Programación 1 TUP-UTN

#Se define la función mediante “def” seguido de su nombre, adentro del paréntesis se ingresan los parámetros (datos de entrada)

```
def suma(a,b):
```

#se escribe el bloque de código que realiza la acción

```
    suma= a+b
```

#al finalizar, se puede retornar el valor deseado con la sentencia “return”

```
    return suma
```

Ordenamientos

En Python se implementa de manera nativa el “ordenamiento” a través de la función `sorted()` y el método `sort()`. Su función es reorganizar una lista global de elementos de forma ascendente o descendente según el criterio elegido, como ejemplo el ordenar los países de forma ascendente por su nombre, el proceso manipula los elementos que contiene lista para optimizar la estructura y presentación de esos datos. Se utiliza para facilitar la lectura y comprensión de la información por parte del usuario final.

Ejemplo:

```
numeros= [40,10,30,20] # Lista desordenada de números
```

#Se usa la función de ordenamiento de forma ascendente

```
numeros_ordenados= sorted(numeros)
```

```
print(numeros_ordenados) #imprime por pantalla: [10,20,30,40]
```

Estadísticas Básicas

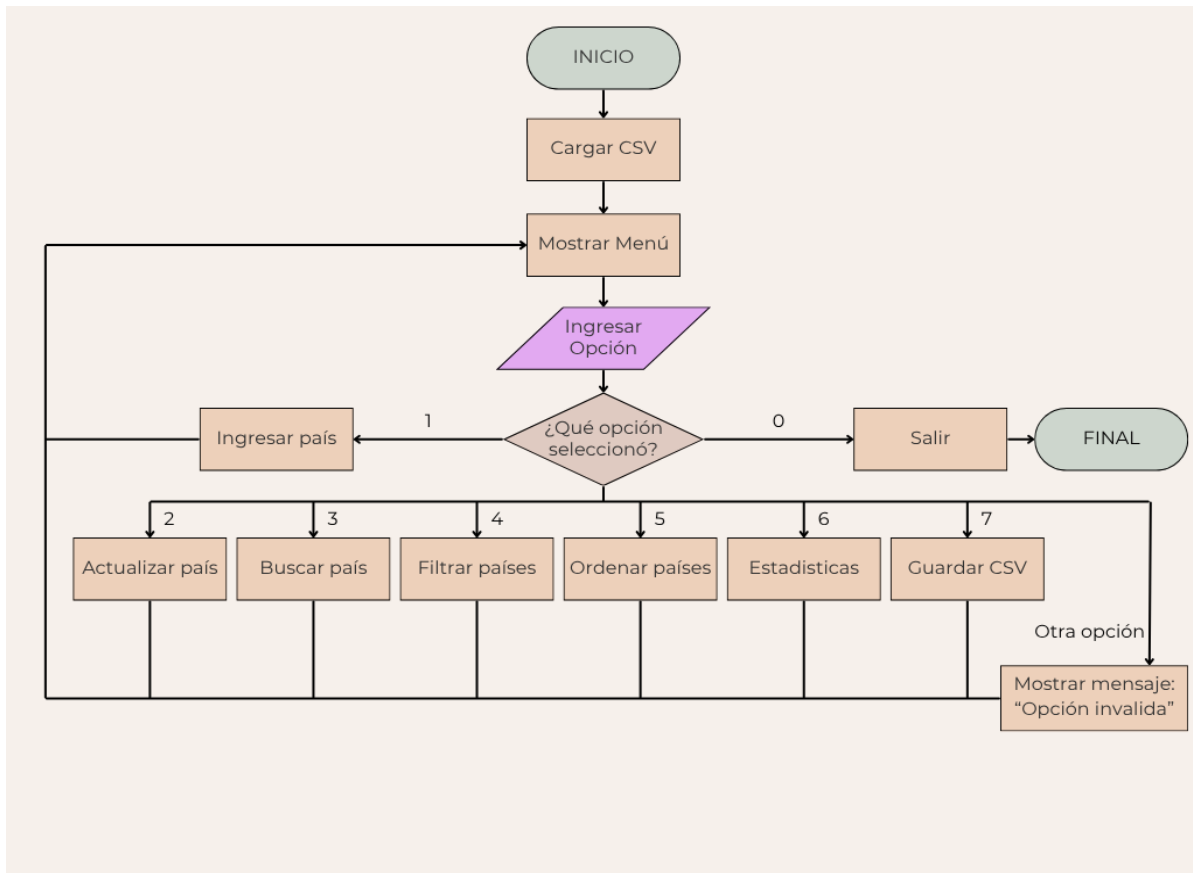
Es la aplicación lógica y matemática sobre un conjunto de datos homogéneo. Se fundamenta en recorrer conjuntos de datos utilizando bucles junto a patrones de diseño como acumuladores (para sumar), contadores (para la frecuencia) y condicionales. Su principal función es transformar los datos brutos en información estadística de valor, esto permite identificar valores máximos y mínimos, promedios y la cantidad de repeticiones de una categoría.

Manejo de archivos (CSV)

En este apartado se realiza el proceso de interacción con el almacenamiento no volátil del sistema operativo mediante objetos de archivo, permitiendo que los datos sobrevivan en el disco duro aun cuando el programa haya finalizado. En este caso, el formato CSV, consiste en abrir e interpretar secuencialmente línea por línea de texto estructuradas en filas y columnas separadas por comas, procesando y convirtiendo dichas cadenas de caracteres en tipo de datos nativos correspondientes del lenguaje, en este caso Python.

Decisiones Técnicas y Arquitectura

Diagrama de flujo del funcionamiento del proyecto:



Estructuras de datos principal

Se utilizó una lista que contiene diccionarios para almacenar los datos

```
países = [ {"nombre": "Argentina", "poblacion": 45376763, "superficie": 2780400, "continente": "América"}, {"nombre": "Japón", "poblacion": 125800000, "superficie": 377975, "continente": "Asia"} ]
```

Capturas de ejecución de ejemplo:

```
? === GESTION DE PAISES === (Use arrow keys)
» 1. Agregar país
   2. Actualizar país
   3. Buscar país
   4. Filtrar países
   5. Ordenar países
   6. Estadísticas
   7. Guardar cambios
   0. Salir
```

1_Interfaz de menú interactivo

```
? === GESTION DE PAISES === (Use arrow keys)
» 1. Agregar país
   2. Actualizar país
   3. Buscar país
   4. Filtrar países
   5. Ordenar países
   6. Estadísticas
   7. Guardar cambios
   0. Salir
```

2_Agregar país

```
== ACTUALIZAR PAÍS ==
País a actualizar: argentina
Nueva población: 10000000
Nueva superficie: 3223232
País actualizado.
Presione cualquier tecla para continuar
```

3_Actualizar país

```
== BUSCAR PAÍS ==
Buscar: ar
- País: Argentina | Poblacion: 10000000 | Superficie: 3223232 | Continente: America
- País: Argelia | Poblacion: 48028300 | Superficie: 2381741 | Continente: Africa
Presione cualquier tecla para continuar
```

4_Buscar país

```
? === FILTROS === (Use arrow keys)
» 1. Continente
   2. Población
   3. Superficie
   0. Volver

? === FILTROS === 1. Continente
== FILTRAR CONTINENTE ==
Continente: america
[Países del continente america]
- País: Argentina | Poblacion: 10000000 | Superficie: 3223232
- País: Brasil | Poblacion: 213993437 | Superficie: 8515767
- País: Chile | Poblacion: 19116201 | Superficie: 756102
Presione cualquier tecla para continuar
```

5_Filtrar

```
? === ORDENAMIENTOS === (Use arrow keys)
» 1. Nombre
   2. Población ascendente
   3. Población descendente
   4. Superficie ascendente
   5. Superficie descendente
   0. Volver

? === ORDENAMIENTOS === 1. Nombre
- País: Alemania | Poblacion: 83149300 | Superficie: 357022 | Continente: Europa
- País: Argelia | Poblacion: 48028300 | Superficie: 2381741 | Continente: Africa
- País: Argentina | Poblacion: 10000000 | Superficie: 3223232 | Continente: America
- País: Australia | Poblacion: 26600000 | Superficie: 7692024 | Continente: Oceania
- País: Brasil | Poblacion: 213993437 | Superficie: 8515767 | Continente: America
- País: Chile | Poblacion: 19116201 | Superficie: 756102 | Continente: America
- País: China | Poblacion: 1412600000 | Superficie: 9596961 | Continente: Asia
- País: España | Poblacion: 47450795 | Superficie: 505990 | Continente: Europa
- País: Francia | Poblacion: 68042591 | Superficie: 551695 | Continente: Europa
- País: India | Poblacion: 1428627663 | Superficie: 3287263 | Continente: Asia
- País: Japon | Poblacion: 125800000 | Superficie: 377975 | Continente: Asia
Presione cualquier tecla para continuar
```

6_Ordenamiento de países

```
== ESTADISTICAS ==
• Mayor población: India
• Menor población: Argentina
• Promedio población: 316673480.64
• Promedio superficie: 3385979.27
• Países por continente:
  - America: 3
  - Europa: 3
  - Asia: 3
  - Oceanía: 1
  - Africa: 1
Presione cualquier tecla para continuar
```

7_Estadisticas

```
== AGREGAR PAÍS ==
Nombre:
Error: No se permiten espacios vacíos
Nombre: 123
Error: Debe ingresar solamente letras
Nombre: 
```

8_Ejemplo de los errores posibles que cometa el usuario

Implementación de interfaces interactivas: Questionary

La librería Questionary, es una librería de terceros diseñada para la creación de interfaces interactivas y visualmente atractivas. Su funcionamiento es el solicitar información al usuario a través de la terminal de una manera amigable, permite crear menús, selecciones, ingreso de rutas de archivos, confirmar acciones destructivas, y mucho más.

Gracias a ser visualmente más atractiva, no frustra o confunde al usuario al colocar manualmente las opciones en un menú, siendo mayor la equivocación del usuario al colocar dicha opción a acceder, en questionary quita esa acción para cambiarlo a usar las flechas del teclado, siendo una experiencia mucho más amigable.

Ejemplo de uso:

```
import questionary

opcion = questionary.select(
    message= "Seleccioná:",
    choices=["Opcion A", "Opcion B", "Salir"]
).ask()

print(f"Elegiste: {opcion}")
```

Dificultades y Conclusiones

Dificultades Técnicas

En el transcurso de la resolución se presentaron diversos desafíos, tales como en el manejo de errores durante la lectura del archivo CSV, específicamente el validar que los campos numéricos de población y superficie no contuvieran caracteres o espacios vacíos, controlar las búsquedas parciales para que no fueran sensibles a mayúsculas y minúsculas. Como también se necesitó especial atención al desarrollo de las funciones específicas para filtración de rango numérico y búsqueda parcial de países.

Conclusión

Para concluir la realización de este trabajo integrador, me permitió poder adquirir una mayor lógica en cuanto a programación estructurada en Python. El uso de combinación de listas y diccionarios demostró ser bastante práctica y ser una solución robusta para la manipulación de datos en memoria, mientras que la modularización facilitó la organización del trabajo y la prueba de los mismo individualmente.

Bibliografía / Webgrafía

<https://questionary.readthedocs.io/en/stable/>

<https://docs.python.org/es/3.14/tutorial/index.html> (Pagina oficial de documentación de Python traducida al español)

Downey, A. B. (2016). *Think Python: How to think like a computer scientist* (2.ª ed.). O'Reilly Media.

<https://github.com/espinoza/ThinkPython2-spanish/blob/master/book/thinkpython2-spanish.pdf> (Versión en español del libro “Think Python (2.ª ed) O'Reilly Media”)

Repositorio de GitHub y video explicativo del proyecto

GitHub: <https://github.com/Limebur/UTN-TPI-Grupo-BB>

Vídeo: